
ecdna-bench

A complete guide — from never having used a computing cluster, to reproducing every number in the paper

Benchmarking six computational methods for detecting and counting extrachromosomal DNA in fluorescence microscopy images

Brunk Lab · University of North Carolina at Chapel Hill

Written for UNC Longleaf · No prior experience assumed

What this is. A hands-on tutorial for the ecdna-bench project: a benchmark of six computational methods for detecting and counting extrachromosomal DNA (ecDNA) in fluorescence microscopy images.

Who it is for. Anyone. No experience with Linux, clusters, Python or image analysis is assumed. If you have used a computer and can copy and paste, you can follow this.

How to use it. Read Part 1 and Part 2 to understand what you are doing. Then work through Part 3 by copying each block into your terminal in order. **You will only ever type one thing yourself: your ONYEN.** Everything after that is copy and paste.

A note on copying from this PDF. PDF viewers sometimes drop the line breaks when you copy several lines at once. Part 3 opens with a script that avoids the problem entirely, and the manual steps include checks that catch it. If you have the Markdown version of this guide, copy from that instead.

How long. About two hours for the setup, most of it spent waiting for downloads. After that you can run any part of the analysis in minutes.

Contents

Part 1 — The tools (*read once, before you start*) 1.1 The terminal · 1.2 Commands · 1.3 Files and paths · 1.4 What a cluster is · 1.5 SLURM · 1.6 Environments · 1.7 Git · 1.8 Notebooks

Part 2 — The science (*read once*) 2.1 ecDNA · 2.2 FISH imaging · 2.3 The dataset · 2.4 The six methods · 2.5 The metrics · 2.6 The pipeline

Part 3 — Setup (*copy and paste, in order*) Steps 1-11

Part 4 — Running the analysis

Part 5 — Notebooks and figures

Part 6 — Worked examples (*the two questions reviewers ask*)

Part 7 — When something goes wrong

Part 8 — Reference

Part 1 — The tools

Six ideas. Once these click, everything else is detail.

1.1 The terminal, the shell, and bash

When you connect to a computing cluster you do not get a desktop. You get a window with text in it and a blinking cursor. There are no icons and the mouse does almost nothing.

That window is called a **terminal**. It is just a place where text goes in and text comes out.

Sitting behind the terminal is a program that reads what you type, works out what you meant, and does it. That program is called a **shell**. There are several shells; the one almost everyone uses is called **bash** (short for "Bourne Again SHell" — a pun on the name of an older shell).

So:

- **Terminal** = the window
- **Bash** = the program inside it that listens to you
- **Command** = one instruction you type and then press Enter

A useful comparison: in Windows or macOS you open a folder by double-clicking it. In bash you open a folder by typing `cd foldername`. Same action, different interface. The reason scientists use the text version is that a typed command can be saved, shared, repeated exactly, and put in a document like this one. A sequence of mouse clicks cannot.

Bash forgets everything between sessions. Close the terminal, open it again, and it starts fresh with no memory of what you did. This matters, and Step 4 of Part 3 is about working around it.

1.2 What a command looks like

Every command has the same shape:

```
program    options    what to act on
```

For example:

```
ls -l /proj
```

- `ls` is the program. It means **list** — show me what is in a folder.
- `-l` is an option. The dash tells bash "this is a setting, not a filename". Here it means "long format": show sizes and dates as well as names.
- `/proj` is the thing to act on — the folder to list.

Some commands you will meet in this guide:

Command	Meaning	Example
<code>pwd</code>	P rint W orking D irectory — where am I?	<code>pwd</code>
<code>ls</code>	List what is here	<code>ls</code>
<code>cd</code>	C hange D irectory — go somewhere	<code>cd /proj</code>
<code>mkdir</code>	M ake d irectory — create a folder	<code>mkdir myfolder</code>
<code>cat</code>	Show the contents of a text file	<code>cat notes.txt</code>
<code>nano</code>	Open a simple text editor	<code>nano notes.txt</code>
<code>cp</code>	C opy a file	<code>cp a.txt b.txt</code>
<code>echo</code>	Print something back to me	<code>echo hello</code>
<code>grep</code>	Search inside files for text	<code>grep error log.txt</code>

Two conventions used throughout this guide:

- **\$ at the start of a line in an example** is the prompt — the thing bash prints to show it is ready. You do not type it. In the code blocks below there is no `$`, so you can safely copy the whole block.
- **# starts a comment.** Bash ignores everything after it on that line. Comments in the blocks below are explanations for you, not instructions for the computer. They are safe to copy.

If you get stuck mid-command and the prompt looks wrong or is waiting for something, press `Ctrl-C` to cancel and get back to a clean prompt.

1.3 Files, folders and paths

Everything on a Linux machine is a file, and files live in folders (also called directories). Folders live inside other folders, all the way up to a single starting point called `/`, pronounced "root".

A **path** is an address. `/proj/brunk_ecdna_cv_project/Poorya/ecdna-bench` means: start at root, go into `proj`, then into `brunk_ecdna_cv_project`, then `Poorya`, then `ecdna-bench`.

Two kinds of path:

- **Absolute** — starts with `/`. Works from anywhere. Like a full postal address.
- **Relative** — does not start with `/`. Interpreted from wherever you currently are. Like saying "two doors down".

Two special shortcuts:

- `.` means "here"
- `..` means "the folder above this one", so `cd ..` goes up one level
- `~` means "my home folder"

Hidden files. Any file whose name begins with a dot is hidden — `ls` will not show it. `ls -a` shows everything including hidden files. This matters because the configuration file you will edit, `.bashrc`, is hidden. It is not secret; the dot is simply a convention meaning "configuration, not content".

1.4 What a computing cluster is

Longleaf is not one computer. It is several hundred computers in a machine room at UNC, sharing one filesystem, used by thousands of people at once.

You interact with two kinds of machine:

Login nodes. When you connect, you land here. There are only a few of them and everyone shares them. They are for editing files, looking at results, and saying "please run this". **They are not for doing real work.** Running something heavy on a login node slows the machine down for everyone and will get you a polite email from the systems team.

Compute nodes. The hundreds of large machines where real work happens. You cannot use one directly. You *ask* for one, and a scheduler gives you one when it is free.

The mental model: the login node is the front desk. The compute nodes are the laboratory. You go to the front desk, fill in a request saying what you need and for how long, and the receptionist tells you when a bench is free.

1.5 SLURM — asking for a compute node

SLURM is the receptionist. It is the program that decides who gets which compute node and when.

You describe your job in a small text file: how many processors, how much memory, how long, and what to run. Then you hand that file to SLURM with the `sbatch` command. SLURM puts you in a queue, and when resources free up it runs your job and writes the output to a log file.

Your job runs whether or not you are still logged in. You can submit a twelve-hour training run, close your laptop, and read the results tomorrow.

The five commands you will actually use:

Command	Meaning
<code>sbatch job.sh</code>	Submit this job. Prints a job number
<code>squeue -u \$USER</code>	What am I running or waiting for?
<code>scancel 12345678</code>	Cancel job 12345678
<code>sacct -j 12345678</code>	Did job 12345678 finish, and how?
<code>seff 12345678</code>	How much memory and CPU did it actually use?

Partitions. Compute nodes are grouped by type. `general` is ordinary processors. `a100-gpu` and `l40-gpu` have graphics cards, which are needed for training neural networks. You state which group you want in the job file.

1.6 Environments — why we do not just "install Python"

Scientific software is a stack of libraries built on other libraries. The analysis in this project uses roughly forty of them: one for reading images, one for statistics, one for neural networks, and so on.

Those libraries change. A function that returns one answer in version 1.2 might return a slightly different answer in version 1.5. If you install the newest version of everything today, you may not reproduce a result computed two years ago.

A **conda environment** solves this. It is a self-contained folder holding its own copy of Python and its own copy of every library, at exactly the versions you specify. Activating an environment tells your terminal "for now, use the Python in this folder". Different environments on the same machine cannot interfere with each other.

This project has a file called `env/environment.yml` that lists every library with an exact version number:

```
- python=3.10.16
- numpy=2.2.5
- scipy=1.15.3
- pandas=2.2.3
```

That file is the recipe. Anyone, on any machine, in any year, can rebuild the same environment from it and get the same numbers. That is what "reproducible" means in practice, and it is what a journal reviewer is really asking about.

A note on the two environments you may hear about. An older environment called `ecDNA_Hybrid` was used to draw the figures. `env/environment.yml` was built by extracting the exact versions out of it, so the two are equivalent — this has been verified numerically, and the details are in §8.3. You need only the environment built from the file.

1.7 Git — the undo button

Git records the exact state of a folder every time you tell it to. Each recorded state is called a **commit**, and each commit is permanent. You can return to any of them at any time.

That gives you a genuine undo button that spans months. If a file gets overwritten by accident, one command brings it back exactly as it was.

GitHub is a website that stores a copy of your git history online, so it survives your laptop and other people can get it.

Four commands cover almost everything:

Command	Meaning
<code>git status</code>	What has changed since the last commit?
<code>git log --oneline -5</code>	Show the last five commits
<code>git restore <file></code>	Undo my changes to this file
<code>git pull</code>	Fetch the latest version from GitHub

You will mostly use `git status` (to check nothing is broken) and `git restore` (to fix it when it is).

1.8 Jupyter notebooks and kernels

A **Jupyter notebook** is a document that mixes text, code and results. You write a small block of code, press Shift-Enter, and the output — a number, a table, a figure — appears directly underneath. Then you write the next block.

It suits data analysis because you can look at the result of each step before deciding what to do next, and because the finished notebook is a readable record of exactly how a figure was made.

A **kernel** is the engine that runs the code. When you open a notebook you choose which kernel to use, and that choice decides which environment — and therefore which library versions — the code runs in.

This matters more than it sounds. Choosing the wrong kernel means running your analysis with different library versions from the ones that produced the published numbers. Step 8 of Part 3 sets up the correct kernel, and after that you select it once per notebook.

Part 2 — The science

You do not need this to run the code, but the code makes far more sense with it.

2.1 What ecDNA is, and why counting it matters

Human DNA normally lives on 46 chromosomes. In many cancers, fragments of DNA break away and form small circles that float free in the nucleus. These are **extrachromosomal DNA**, or **ecDNA**.

They matter for three reasons:

They carry oncogenes. The circles frequently contain the genes driving the cancer. A cell with fifty copies of a growth-signalling gene grows far more aggressively than one with two.

They are inherited unevenly. Chromosomes are shared out precisely when a cell divides. ecDNA circles are not — they scatter more or less at random. One daughter cell may get sixty copies and the other ten. Over many divisions this generates enormous diversity within a single tumour.

That diversity causes drug resistance. When a drug is applied, most cells die, but a few happened to inherit a copy number that lets them survive. They repopulate the tumour. ecDNA is one of the main reasons cancers come back.

So the biologically meaningful quantity is **how many ecDNA copies are in each individual cell** — not an average across a population. That means counting them, cell by cell, in images.

2.2 How the images are made

FISH stands for **F**luorescence **I**n **S**itu **H**ybridisation. A short piece of synthetic DNA is designed to stick only to the sequence of interest, and a fluorescent dye is attached to it. Under a microscope, every place that sequence exists lights up as a bright dot.

To see the DNA clearly, cells are chemically arrested in **metaphase** — the stage of division when DNA is at its most compact — and then burst open on a slide so their contents spread out. The result is a **metaphase spread**: one cell's worth of chromosomes and ecDNA, laid out flat.

Each image in this dataset has two channels:

- **RGB** — the FISH signal. Chromosomes appear as large elongated shapes; ecDNA appears as small round dots.
- **DAPI** — a stain that binds all DNA, giving overall context.

Images are $2,448 \times 2,048$ pixels. A single image may contain anywhere from zero to more than 1,600 ecDNA dots.

Why this is hard to automate. The dots are small, sometimes only a few pixels. They vary in brightness. They overlap. Debris and neighbouring cells appear in the same field. And in a dense image, hundreds of dots may be packed close together. Counting by eye is slow, tiring and inconsistent between observers, which is exactly why automated methods exist — and exactly why they need to be tested against a careful human standard.

2.3 What is in the dataset

The full resource: 2,986 images across five cancer cell lines NCI-H2170 (lung), SUM159PT (breast), SNU16 (gastric), COLO320DM (colorectal), NCI-H716 (colorectal). Every one of these has an expert's ecDNA annotations — a human marked the position of every dot.

The benchmark: 1,145 of those images These carry one extra thing: a hand-drawn **ROI mask**. ROI means Region Of Interest — an outline around the single metaphase spread that should be analysed, excluding neighbouring cells and debris. Drawing them is slow, which is why only a subset has them. SUM159PT has none, which makes it a genuinely unseen cell line for testing.

The split: 800 training / 170 validation / 175 test A standard and important division:

- **Training** — the images a learning method is allowed to see and learn from.
- **Validation** — used during development to check progress and choose settings. Seen often, but never learned from directly.
- **Test** — locked away and used exactly once, at the end. It is the only honest estimate of performance on new data.

Every image also carries a **ground-truth mask**: a small marker drawn at each position a human said contained an ecDNA. This is the standard everything is measured against.

2.4 The six methods being compared

Method	What it is
Classic (after opt)	A traditional image-processing pipeline — enhance contrast, threshold, find blobs — with its settings tuned automatically per cell line
Label Engine	A published commercial/academic segmentation tool
ecSeg	A published deep-learning tool built specifically for ecDNA
MIA	Another published image-analysis tool
ecCount (threshold mask)	The model developed in this project, read out by thresholding its output
ecCount (peaks)	The same model, read out by finding local peaks — the headline method

The last two are two different ways of interpreting the output of a single neural network. The network produces a probability map — a picture where brightness means "how likely is there an ecDNA here". You can turn that into counts either by keeping everything above a brightness cut-off (threshold), or by finding the local high points (peaks). Peaks separates touching dots better, and performs better.

2.5 How performance is measured

Three numbers, each answering a different question.

Object F1 — did you find the right dots?

Every predicted dot is matched to a true dot if they are close enough and overlap enough. Then:

- **Precision** = of the dots you reported, what fraction were real? (Low precision = you invent things.)
- **Recall** = of the real dots, what fraction did you find? (Low recall = you miss things.)
- **F1** combines the two into one number between 0 and 1. Higher is better.

Count MAE — how far off is the number?

Mean Absolute Error. If the truth is 200 and a method says 170, the error is 30. Average that across all images. Lower is better. Reported in ecDNA per image.

Signed bias — does it lean one way?

The same errors, but keeping the sign. If a method always undercounts, its bias is negative. A method could have a large MAE but zero bias if it overshoots and undershoots equally.

Why bias is the one biologists should care about. A method with a bias of -40 does not simply report smaller numbers. It reports smaller numbers *by an amount that depends on how crowded the image is* — the more dots there are, the more it loses. That distorts comparisons between conditions. In this dataset it happens in practice: for one drug treatment, the ground truth shows no change in ecDNA burden, and so does ecCount, while three other methods report a statistically significant decrease. Same images, opposite biological conclusion.

The results

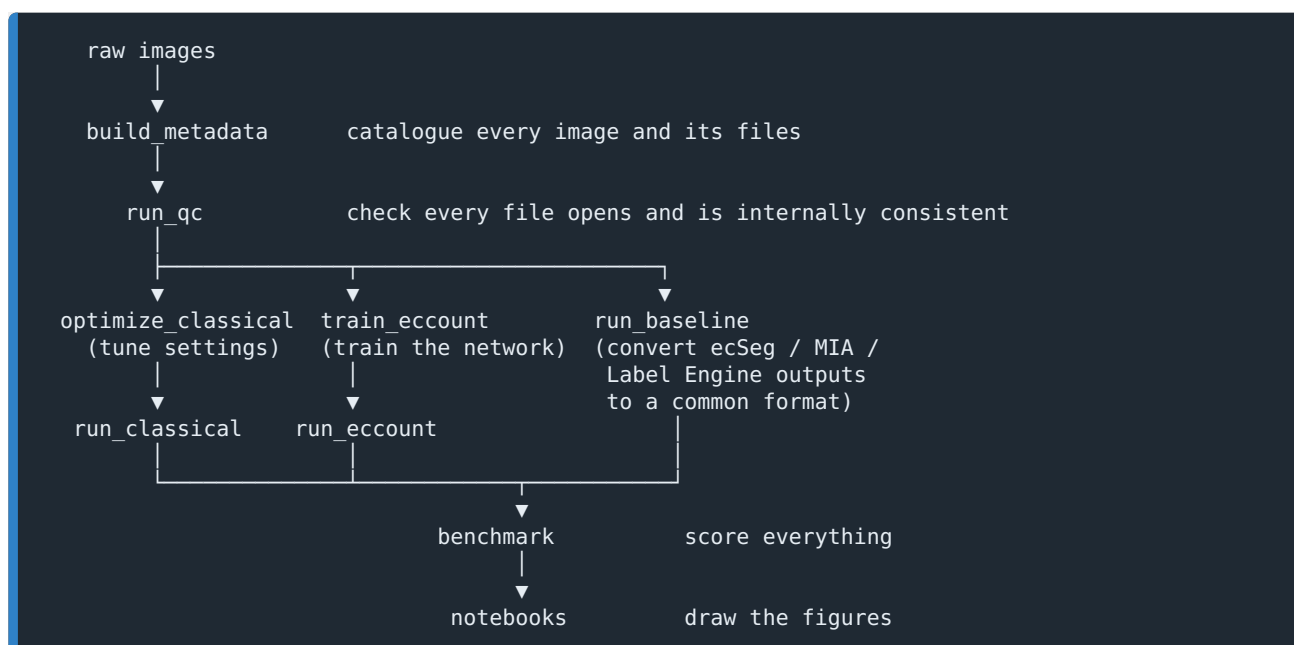
Across all 1,145 benchmark images, OR matching:

Method	Object F1	Count MAE	Signed bias
ecCount (peaks)	0.942	13.1	+0.4
ecCount (threshold mask)	0.917	18.1	-11.4
Label Engine	0.825	36.0	-28.8
MIA	0.800	47.0	-40.6
Classic (after opt)	0.777	44.9	-24.3
ecSeg	0.464	121.5	-118.5

The bias column is the striking one. Every method except ecCount (peaks) systematically undercounts, some severely.

One piece of jargon you will meet: OR matching. To decide whether a predicted dot corresponds to a true dot, you can require that they are close together **or** that they overlap (OR), or insist on **both** (AND). This project uses OR throughout, and every published number comes from the `or_matching/` folder. Results computed with AND exist in a sibling folder for a supplementary sensitivity check. **The two folders contain files with identical names.** Reading from the wrong one gives plausible but wrong numbers. Always check which folder you loaded from.

2.6 The pipeline



Each stage reads files and writes files. You can run any stage on its own as long as the files it needs already exist. In practice you will rarely run more than one or two.

Part 3 — Setup

The quick route — one command

Everything in this Part can be done by a single script that ships with the project. If you have already logged in (Step 1 below), run:

```
bash /proj/brunk_ecdna_cv_project/Poorya/ecdna-bench/scripts/setup_new_user.sh
```

It asks for your ONYEN, then does Steps 2 to 11 for you and checks its own work. It is safe to run more than once — anything already done is skipped.

Use the script if you just want to get working. Read the rest of this Part if you want to understand what it did, or if the script reports a problem.

Why a script and not copy-paste? Copying a multi-line block out of a PDF sometimes loses the line breaks. When that happens to a block of `export` commands, bash accepts it without an error but sets the wrong values — and you find out much later, with output in the wrong folder. The script cannot suffer from this. If you do paste manually, run the checks at the end of each step; they are designed to catch exactly this failure.

The manual route

From here on, copy each block into your terminal and press Enter.

You will type exactly one thing yourself, in Step 2: your ONYEN. Everything after that uses it automatically.

Do the steps in order. Each one ends with a check so you know it worked before moving on. If a check fails, Part 7 has the fix.

Step 1 — Log in

You need a Longleaf account and membership of the `brunk_ecdna_cv_project` group. Both are requested at <https://help.rc.unc.edu>. If you already log in to Longleaf you have the account; ask Research Computing to add you to the group.

On macOS or Linux: open the Terminal application. **On Windows:** open PowerShell, or Windows Terminal.

Then type this, replacing `YOUR_ONYEN` with your UNC username:

```
ssh YOUR_ONYEN@longleaf.unc.edu
```

Enter your ONYEN password (the screen shows nothing as you type — this is normal), then approve the Duo prompt on your phone.

Check. Your prompt should now look something like:

```
[yourname@longleaf-login2 ~]$
```

The `longleaf-login2` part confirms you are on the cluster. To leave at any time, type `exit`.

Step 2 — Tell the guide who you are

This is the only line in the entire guide you edit. Replace `REPLACE_WITH_YOUR_ONYEN` with your actual ONYEN — for example `jsmith` — then copy the whole block:

```
export ONYEN=REPLACE_WITH_YOUR_ONYEN;
export PROJ=/proj/brunk_ecdna_cv_project;
export MYDIR=$PROJ/$ONYEN;
export REPO=$MYDIR/repos/ecdna-bench;
export ENV_CANONICAL=$PROJ/Poorya/envs/ecdna-bench;
export DATA=$PROJ/Poorya/ecDNA_Data;
export SOURCE_REPO=$PROJ/Poorya/ecdna-bench;
export ECDNA_PYTHON=$ENV_CANONICAL/bin/python;
```

The semicolons are deliberate. If your clipboard flattens this block onto one line — which happens when copying from a PDF — the semicolons keep bash running each assignment separately and in order. Without them, bash would accept the flattened line silently and build every path from an empty value.

What just happened. Each `export NAME=value` creates a named shortcut called an **environment variable**. From now on, typing `$PROJ` anywhere means bash substitutes `/proj/brunk_ecdna_cv_project` before running the command. It is exactly like saving a contact in your phone: you type the name, the phone dials the number.

The word `export` means "make this available to other programs I run", not just to bash itself.

Check — and read the output, do not just glance at it:

```
echo "ONYEN      : $ONYEN"
echo "My folder  : $MYDIR"
echo "Jobs will use : $ECDNA_PYTHON"
```

Three ways this goes wrong, and how to recognise each:

What you see	What happened	Fix
<code>ONYEN : REPLACE_WITH_YOUR_ONYEN</code>	You did not edit the first line	Edit it, paste again
<code>My folder : /yourname — missing / proj/...</code>	The paste lost its line breaks. Every path was built before <code>PROJ</code> existed	Paste the lines one at a time
<code>My folder : /proj/ brunk_ecdna_cv_project/yourname</code>	Correct	Continue

The second one is the dangerous case: bash reports no error, and you would not discover the problem until output appeared somewhere unexpected. Always read this check.

These shortcuts vanish when you close the terminal. That is what Step 4 fixes. Until then, if you disconnect, re-run this block.

Step 3 — Understand where things live

Longleaf gives every user several storage areas, each with a different purpose, size limit and lifetime. Getting these wrong is the most common source of confusion.

```
# Show them all with sizes and how full they are
df -h $HOME /work/users/${ONYEN:0:1}/${ONYEN:1:1}/${ONYEN} $PROJ 2>/dev/null
```

The areas, and what each is for:

Area	Path	Size	Use it for
Home	<code>/nas/longleaf/home/\$ONYEN</code>	~50 GB	Settings and small text files only
Home, alias	<code>/users/b/e/\$ONYEN</code>	same place	The same folder under a shorter name
Scratch	<code>/work/users/b/e/\$ONYEN</code>	~10 TB	Temporary files. Deleted automatically. Never keep anything here
Lab project	<code>/proj/ brunk_ecdna_cv_project</code>	large	Everything real. Data, environments, results
Data commons	<code>/datacommons</code>	—	Shared public datasets. Not used here

Why does home have two names? The `b/e` in `/users/b/e/behnamie` is just the first two letters of the ONYEN. With tens of thousands of accounts, putting them all in one folder would be unmanageable, so they are filed into sub-folders by initial letters — like a filing cabinet with A–Z dividers.

You can prove the two names point at the same folder:

```
echo "HOME is set to: $HOME"
readlink -f /nas/longleaf/home/$ONYEN
readlink -f /users/${ONYEN:0:1}/${ONYEN:1:1}/$ONYEN
```

`readlink -f` follows any shortcuts and prints the one true location. If the last two lines match, they are the same folder under two names.

The rule to remember: home is small, scratch is temporary, `/proj` is where work lives.

Step 4 — Make your settings permanent

Bash forgets everything when you close the terminal. To avoid retyping Step 2 every session, we write those lines into a file bash reads automatically.

What `.bashrc` is. A plain text file in your home folder. Every time bash starts, it reads this file and runs every line in it, exactly as if you had typed them. The leading dot means the file is hidden from ordinary listings — a convention meaning "configuration, not content". Nothing about it is secret or dangerous.

First, make a backup. If anything goes wrong you can restore it:

```
cp ~/.bashrc ~/.bashrc.backup-$(date +%Y%m%d)
ls -l ~/.bashrc*
```

Now add the settings. This block appends to the file rather than replacing it, so anything already there is preserved:

```
cat >> ~/.bashrc <<EOF

# =====
# ecdna-bench project settings — added $(date +%Y-%m-%d)
# =====

# Shortcuts, so long paths become short names
export ONYEN=$ONYEN
export PROJ=/proj/brunk_ecdna_cv_project
export MYDIR=\$PROJ/\$ONYEN
export REPO=\$MYDIR/repos/ecdna-bench
export ENV_CANONICAL=\$PROJ/Poorya/envs/ecdna-bench
export DATA=\$PROJ/Poorya/ecDNA_Data
export SOURCE_REPO=\$PROJ/Poorya/ecdna-bench

# Keep downloaded package caches off the small home quota
export PIP_CACHE_DIR=\$MYDIR/.cache/pip
export XDG_CACHE_HOME=\$MYDIR/.cache
export CONDA_PKGS_DIRS=\$MYDIR/.cache/conda/pkgs

# REQUIRED for cluster jobs: which Python the job scripts should use
export ECDNA_PYTHON=\$ENV_CANONICAL/bin/python

# One-word shortcut to switch on the project environment
alias ecdna='module load anaconda && conda activate \$ENV_CANONICAL && cd \$REPO'
EOF
```

What each part does:

- **The shortcuts** are the same ones from Step 2, now permanent.

- **The cache lines** are the important ones. When conda or pip downloads a package, it keeps a copy so the next install is faster. By default those copies pile up in your home folder, which has a 50 GB limit. When home fills up, Jupyter stops being able to write its session files and simply refuses to start, usually with an error message that mentions nothing about disk space. These three lines send the caches to `/proj` instead, which has terabytes. Doing this *before* installing anything avoids the problem entirely.
- **ECDNA_PYTHON is required for cluster jobs.** Job scripts cannot use `conda activate` — activating an environment inside a batch job conflicts with how the cluster sets up its command paths. Instead every job script looks for this variable and uses the Python it points at. Without it, jobs fall back to the system Python and fail immediately with `ModuleNotFoundError: No module named 'ecdna_bench'`. This one line prevents the most common cluster failure in this project.
- **The alias** turns four commands into one word. Typing `ecdna` will load the module system, switch on the environment, and move you into the project folder.

Apply the changes without logging out — `source` means "read this file and run it now":

```
source ~/.bashrc
```

Check:

```
echo "ONYEN          : $ONYEN"
echo "My folder      : $MYDIR"
echo "Cache goes to   : $PIP_CACHE_DIR"
echo "Jobs will use    : $ECDNA_PYTHON"
ls -l "$ECDNA_PYTHON"
```

All four should print sensible values with no `REPLACE_WITH` left anywhere, and the final `ls` must find the file. If `ECDNA_PYTHON` is empty or the file is missing, every cluster job you submit will fail.

If you ever need to undo this: `cp ~/.bashrc.backup-YYYYMMDD ~/.bashrc` using the date from your backup, then `source ~/.bashrc`.

Step 5 — Create your workspace

Somewhere for your own work, separate from anyone else's:

```
mkdir -p $MYDIR/{repos,runs,logs,envs,.cache}
ls -la $MYDIR
```

What the curly braces do. `{repos,runs,logs}` is bash shorthand that expands into three separate names, so one command creates all five folders. `-p` means "create parent folders if needed, and do not complain if it already exists".

What each is for:

Folder	Contents
<code>repos/</code>	Copies of code
<code>runs/</code>	Everything you produce — results, trained models, figures
<code>logs/</code>	Output from cluster jobs
<code>envs/</code>	A personal environment, if you build one
<code>.cache/</code>	Downloaded packages (from Step 4)

Step 6 — The golden rule

Read from Poorya's folders. Write only into your own.

The published results live in `$SOURCE_REPO/release/`. They are what the paper cites. Nothing you run should ever write there.

Three independent layers make this safe, so a mistake is never fatal:

1. **Git** — every result file is committed and tagged. `git restore` brings back anything overwritten, exactly.
2. **Separate copies** — you work in your own clone, so your settings cannot affect anyone else's.
3. **Write protection** — the frozen results folders are set read-only at the filesystem level, so an accidental write fails with `Permission denied` rather than quietly succeeding.

You do not have to remember to be careful. Step 9 sets up a configuration file that sends every output into your own folder automatically.

Step 7 — Get the code

```
cd $MYDIR/repos
git clone https://github.com/PooryaBehnamie/ecdna-bench.git
cd $REPO
```

What this does. `git clone` downloads the complete project — every file plus its entire history — into a new folder called `ecdna-bench`.

(The repository will move to the `brunklab` organisation. After that, use `https://github.com/brunklab/ecdna-bench.git`. Same contents, same history.)

If GitHub access is not set up yet, copy from the cluster instead:

```
cp -r $SOURCE_REPO $MYDIR/repos/ecdna-bench
cd $REPO
```

Check. These show where you are and the three most recent recorded states:

```
pwd
git log --oneline -3
```

You should see your own path, and three lines each beginning with a short code like `2ba29f4`.

Step 8 — Switch on the environment

The environment is already built and readable by everyone in the project group. You do not need to build your own unless you want one.

```
module load anaconda
conda activate $ENV_CANONICAL
```

What `module load` is. Longleaf keeps many versions of many programs installed side by side and gives you none of them by default, so that nothing conflicts. `module load anaconda` means "add conda to my available commands for this session". You need it once per login — and the `ecdna` alias from Step 4 does it for you.

What `conda activate` does. It puts this environment's Python at the front of the queue, so from now on `python` means *that* Python, with those exact library versions.

Check. Your prompt should now start with `(ecdna-bench)`, and:

```
which python
python --version
python -c "import ecdna_bench; print('project code found')"
```

Expected:

```
/proj/brunk_ecdna_cv_project/Poorya/envs/ecdna-bench/bin/python
Python 3.10.16
project code found
```

From your next login onwards, all of Step 8 is just: `ecdna`

Step 9 — Make the notebooks work

Jupyter cannot see conda environments by itself. You register this one once, and afterwards it appears in the notebook menu.

```
python -m ipykernel install --user \
    --name ecdna-bench \
    --display-name "ecdna-bench (canonical)"
```

What a kernel is. The engine that runs the code inside a notebook. Registering the environment as a kernel means a notebook can run its code using these exact library versions — the ones that produced the published numbers.

Check:

```
jupyter kernelspec list
```

`ecdna-bench` should appear in the list.

Step 10 — Point the code at the data

The project ships `configs/default.yaml`, which holds every published setting. **Never edit that file.** Machine-specific paths go in a separate file that is merged over it automatically.

The block below creates that file with the correct contents. Copy it whole:

```
cd $REPO

cat > configs/paths.local.yaml <<EOF
# =====
# Local paths – generated $(date +%Y-%m-%d) for $ONYEN
# INPUTS point at the shared data (read only).
# OUTPUTS point at this user's own folder.
# =====
paths:
  # ---- inputs: shared, never written to ----
  data_root:      $DATA
  metadata_csv:   $SOURCE_REPO/release/manifests/metadata.csv
  consistency_csv: $SOURCE_REPO/release/manifests/
  dl_master_metadata_stage1_step3_consistency.csv

  splits:
    train: release/split_files/train_ids.csv
    val:   release/split_files/val_ids.csv
    test:  release/split_files/test_ids.csv

  # ---- outputs: everything below lands in your own folder ----
  results_root:  $MYDIR/runs/results
  logs_root:     $MYDIR/runs/logs
  ecount_out_dir: $MYDIR/runs/eccount_training
EOF

cat configs/paths.local.yaml
```

What a YAML file is. A settings file written to be readable by humans. Indentation shows what belongs to what — `data_root` is indented under `paths`, so its full name is `paths.data_root`. **Indentation must be spaces, never tabs.**

Check that the settings loaded correctly:

```
python -c "
from ecdna_bench.config import load_config
c = load_config('configs/default.yaml')
print('settings file used :', c.local_override_yaml)
print('reads data from    :', c.paths.data_root)
print('writes results to  :', c.paths.results_root)
"
```

Three things to confirm:

- `settings file used` must name your `paths.local.yaml`. If it says `None`, the file was not found and your settings are being ignored entirely.
- `reads data from` should be the shared data folder.
- `writes results to` must contain **your** ONYEN.

A trap worth knowing about. There are two functions called `load_config`, and they behave differently:

Import from	Returns	Behaviour
<code>ecdna_bench.config</code>	a typed object	Strict. Read values with a dot: <code>c.paths.results_root</code> . Any <code>paths</code> key it does not declare is silently dropped, so <code>consistency_csv</code> and <code>ecount_out_dir</code> do not appear
<code>ecdna_bench.cli._common</code>	a plain dictionary	Merges your paths file and keeps every key: <code>cfg["paths"]</code> <code>["results_root"]</code>

The command-line tools use the second one, so every key in your `paths.local.yaml` reaches them. The typed loader is a narrower view used elsewhere. Using square brackets on the typed object raises `TypeError: 'Config' object is not subscriptable`; using a dot on the dictionary raises `AttributeError`. If you get either, you have the wrong one of the two.

Step 11 — Prove it all works

Three checks. All three should pass before you run anything real.

Check 1 — the automated tests (about 30 seconds):

```
cd $REPO
pytest -q
```

Expected last line: `124 passed, 2 skipped`.

These tests verify the code's internal logic — matching, metrics, file handling. They need no data, so if they pass, your installation is sound.

Check 2 — the model builds correctly:

```
python -c "
from ecdna_bench.eccount.model import build_model, ModelConfig
m = build_model(ModelConfig())
n = sum(p.numel() for p in m.parameters() if p.requires_grad)
print('trainable parameters:', n)
assert n == 7849601, 'ARCHITECTURE HAS CHANGED'
print('matches the published model exactly')
"
```

Expected: `7849601`, then the confirmation line.

That number is the count of adjustable values inside the neural network. It is checked automatically every time the model is built, so any accidental change to the architecture fails loudly here instead of quietly producing different results.

Check 3 — the graphics card is visible. This one needs a compute node, because login nodes have no graphics card. Request one for twenty minutes:

```
srun --partition=a100-gpu --qos=gpu_access --gres=gpu:1 \
    --mem=16g -t 0:20:00 --pty bash
```

Wait for a new prompt (the machine name changes), then:

```
module load anaconda
conda activate $ENV_CANONICAL
python -c "
import torch
print('graphics card available:', torch.cuda.is_available())
print('card:', torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'none')
"
exit
```

Expected: `True` and a card name such as `NVIDIA A100`. The `exit` returns you to the login node and releases the node for someone else.

If check 3 says `False` while you are still on a login node, that is correct behaviour, not a fault.

Setup complete

You now have: an account, permanent shortcuts, your own workspace, the code, a working environment, a notebook kernel, and a configuration that keeps your output separate from everyone else's.

Every future session starts with one word:

```
ecdna
```

Part 4 — Running the analysis

There are two ways to run things, and choosing correctly matters.

4.1 Two ways to run: directly, or as a cluster job

Directly on the login node. You type a command, it runs, you watch it finish. Fine for anything that takes seconds and uses little memory.

As a cluster job. You hand a job file to SLURM, it queues, and it runs on a proper compute node. Necessary for anything real.

Use cluster jobs for everything except quick checks. Real analysis on a login node is slow, competes with everyone else's work, and gets killed without warning when it uses too much memory. The project ships a ready-made job file for every stage in the `slurm/` folder, and those are the intended way to run the pipeline.

4.2 How the job scripts work

Three things about them, and all three matter.

1. Submit from the repository root, never from inside `slurm/`.

The scripts work out where the project lives from the folder you submitted from. Submit from the wrong place and they look for files that are not there.

```
cd $REPO          # correct
sbatch slurm/submit_benchmark.sh
```

```
cd $REPO/slurm    # WRONG – do not do this
sbatch submit_benchmark.sh
```

2. `logs/` must exist before your first submission.

SLURM writes each job's output into `logs/`. It will not create that folder, and a job whose log destination does not exist fails instantly with no explanation. Create it once:

```
mkdir -p $REPO/logs
```

3. `ECDNA_PYTHON` must be set.

The scripts do not use `conda activate`, because activating an environment inside a batch job conflicts with the cluster's module system. Instead each script reads `ECDNA_PYTHON` and uses that interpreter directly. Step 4 of Part 3 put this in your `.bashrc`; confirm it is live:

```
echo $ECDNA_PYTHON
```

It should print a path ending `envs/ecdna-bench/bin/python`. If it prints nothing, `source ~/.bashrc` and check again.

4.3 The one safety point that matters

Some job scripts write into `release/frozen_results/` by default. `submit_benchmark.sh` and `submit_sensitivity.sh` both do. If you submit them from the original repository, they will attempt to overwrite the published results.

The fix is simple, and it is automatic if you followed Part 3. The scripts determine the project root like this:

```
ECDNA_PROJECT_ROOT → else the folder you submitted from → else the current folder
```

So **if you submit from your own clone, every output lands in your own clone**. That is why Step 7 had you make your own copy.

Two checks before submitting anything:

```
pwd
# must be YOUR repository, containing your ONYEN – not /Poorya/

echo $ECDNA_PROJECT_ROOT
# should be empty (the scripts will use the folder you submit from)
# or your own repository path
```

If you ever need to be explicit:

```
ECDNA_PROJECT_ROOT=$REPO sbatch slurm/submit_benchmark.sh
```

4.4 The job scripts

Script	Runs on	Roughly	What it does
<code>submit_eccount_train.sh</code>	<code>a100-gpu, l40-gpu</code>	12–24 h	Train the network from scratch
<code>submit_eccount_infer.sh</code>	<code>a100-gpu, l40-gpu</code>	1–4 h	Run the trained network on images
<code>submit_benchmark.sh</code>	<code>general</code>	1–4 h	Score every method against ground truth
<code>submit_sensitivity.sh</code>	<code>general</code>	2–8 h	Sweep the matching settings
<code>submit_optimize_classical.sh</code>	<code>general</code>	~6 h	Tune the classical pipeline, one task per cell line
<code>submit_classical_default.sh</code>	<code>general</code>	~1 h	Classical pipeline, untuned settings
<code>submit_classical_optimized.sh</code>	<code>general</code>	~1 h	Classical pipeline, tuned settings
<code>submit_benchmark_classical_before_after.sh</code>	<code>general</code>	~1 h	Compare classical before and after tuning
<code>submit_optimize_classical_h2170_recovery.sh</code>	<code>general</code>	~6 h	A re-run for one cell line
<code>verify_gt_annotation_triplets.slurm</code>	<code>general</code>	short	Check the annotation files are consistent
<code>verify_gt_reconstruction_diagnostics.slurm</code>	<code>general</code>	short	Check ground-truth reconstruction

There is a `slurm/README.md` in the repository with the same table and more detail.

4.5 Your first job

Scoring is the safest thing to run first — it reads predictions that already exist and computes every metric in the paper.

```
cd $REPO
mkdir -p logs
echo $ECDNA_PYTHON          # must not be empty
pwd                          # must be YOUR repository

sbatch slurm/submit_benchmark.sh
```

SLURM prints a job number, for example `Submitted batch job 51234567`. Write it down.

Watch it:

```
squeue -u $USER
```

ST column	Meaning
PD	Pending — waiting for a free node
R	Running
(nothing listed)	Finished, one way or the other

Follow the output as it happens:

```
tail -f logs/benchmark_51234567.out
```

Use your own job number. `Ctrl-C` stops watching; the job keeps going.

When it finishes, check it succeeded:

```
sacct -j 51234567 --format=JobID,JobName,State,Elapsed,ExitCode
seff 51234567
```

`State=COMPLETED` with `ExitCode=0:0` means success. Anything else — see §4.7.

Look at your results:

```
python -c "
import pandas as pd, glob, os
f = glob.glob(os.environ['REPO'] + '/release/frozen_results/or_matching/summary_overall.csv')
d = pd.read_csv(f[0])
print(d[['model', 'obj_f1', 'count_mae', 'count_bias']].round(3).to_string(index=False))
"
```

Compare against the published table in §2.5. If they match, you have just reproduced the paper's headline result on your own account.

4.6 Changing how a job runs

The scripts read settings from the environment, so you can change behaviour without editing any file. Put the setting in front of `sbatch`:

```
# Sweep only one model, on the test images
MODEL=ecount_peaks SPLIT=test sbatch slurm/submit_sensitivity.sh

# Use eight parallel workers and a fixed random seed
N_WORKERS=8 SEED=1234 sbatch slurm/submit_optimize_classical.sh
```

Settings you can override:

Variable	Used by	Meaning
<code>ECDNA_PYTHON</code>	all	Which Python to run
<code>ECDNA_PROJECT_ROOT</code>	all	Where the project lives
<code>MODEL</code>	sensitivity	Which method, or <code>all</code>
<code>SPLIT</code>	sensitivity	<code>train</code> , <code>val</code> , <code>test</code> , <code>all</code>
<code>N_WORKERS</code>	optimisation, benchmark	Parallel workers
<code>SEED</code>	optimisation	Random seed, for reproducibility
<code>LOG_LEVEL</code>	most	<code>INFO</code> or <code>DEBUG</code>
<code>FORCE</code>	sensitivity	<code>1</code> recomputes instead of reusing

To change something the variables do not cover — time limit, memory, partition — copy the script first and edit the copy:

```
cp slurm/submit_eccount_train.sh slurm/submit_eccount_train_mine.sh
nano slurm/submit_eccount_train_mine.sh
sbatch slurm/submit_eccount_train_mine.sh
```

The `#SBATCH` lines at the top are the request: `-t` is the time limit, `--mem` the memory, `-p` the partition. A job is killed the moment it exceeds `-t`, so allow generous headroom.

4.7 If a job fails

Read the `.err` file. The real error is almost always there, not in `.out`.

```
ls -lt logs | head
tail -50 logs/<the newest .err file>
```

State	Meaning	What to do
<code>COMPLETED</code> <code>0:0</code>	Success	—
<code>OUT_OF_MEMORY</code>	Ran out of memory	Copy the script, raise <code>--mem</code>
<code>TIMEOUT</code>	Hit the time limit	Copy the script, raise <code>-t</code>
<code>FAILED</code>	The program errored	Read the <code>.err</code> file
Rejected at submission	Bad partition or QOS name	See below

If submission is rejected, partition names change when a cluster is upgraded. List what exists now:

```
sinfo -s
```

then update the `#SBATCH -p` line in your copy of the script.

4.8 Running things directly, for quick checks

Small, fast commands are fine on a login node.

```
cd $REPO

# Rebuild the image catalogue (~5 min)
python -m ecdna_bench.cli.build_metadata --config configs/default.yaml

# Quality control: does every file open, do the channels match, do the
# recorded counts match the marks in each mask? (~5 min)
python -m ecdna_bench.cli.run_qc --config configs/default.yaml

# Convert the external tools' output into the common format (~5 min total)
python -m ecdna_bench.cli.run_baseline --model ecseg
python -m ecdna_bench.cli.run_baseline --model mia
python -m ecdna_bench.cli.run_baseline --model label_engine
```

Score into a folder of your own, rather than the default location:

```
python -m ecdna_bench.cli.benchmark \
  --config configs/default.yaml \
  --output-dir $MYDIR/runs/benchmark_$(date +%Y%m%d)
```

`--output-dir` is the switch that keeps a run away from anything published. When running by hand, always use it.

Anything longer than about ten minutes belongs in a cluster job.

4.9 About training

Training reads a spreadsheet listing every image with a `split` column saying `train`, `val` or `test`. It learns from the `train` rows, checks itself on the `val` rows, and never touches `test`.

```
cd $REPO
mkdir -p logs
sbatch slurm/submit_eccount_train.sh
```

Twelve hours or so. Watch progress — training writes one row per epoch to a file you can read while it runs:

```
squeue -u $USER
tail -f $MYDIR/runs/eccount_training/train_history.csv
```

That spreadsheet is what makes Part 6 easy. The experiment is defined by a column in a file, not by anything written in the code. Change the column, and you have a different experiment — with no programming at all.

The published run stopped improving at epoch 49, with a best validation loss of 0.6421.

Part 5 — Notebooks and figures

Every figure in the paper is produced by one of five Jupyter notebooks. They are the visual layer on top of the results computed in Part 4.

5.1 Opening a notebook

1. Go to <https://ondemand.rc.unc.edu> and sign in with ONYEN and Duo.
2. **Interactive Apps → Jupyter Notebook**
3. Fill in the form: - **Partition:** `general` - **Hours:** 4 - **CPUs:** 4 · **Memory:** 32 GB
4. Click **Launch**. Your request joins the queue; usually under a minute.
5. When the panel turns green, click **Connect to Jupyter**.
6. Navigate to your repository, then the `notebooks` folder, and open one.
7. **This step matters: Kernel → Change kernel → ecdna-bench (canonical)**

Confirm you are on the right engine. In the first empty cell, type this and press Shift-Enter:

```
import sys, scipy, skimage
print(sys.executable)
print("scipy", scipy.__version__, "| scikit-image", skimage.__version__)
```

The path should contain `envs/ecdna-bench`. If it says `ecDNA_Hybrid` or anything else, go back to step 7.

How to run cells: `Shift-Enter` runs the current cell and moves on. `Ctrl-Enter` runs it and stays. **Kernel → Restart & Run All** runs the whole notebook from a clean start — the only honest way to check a notebook works end to end.

5.2 What each notebook produces

Notebook	Produces
<code>01</code>	Figure 1 — what is in the dataset, and a representative image
<code>02</code>	Figure 2 — how matching works; Figure 3 — the classical pipeline
<code>03</code>	Figure 5 — the ecCount model; Figure 6 — the six-method comparison
<code>04</code>	Supplementary — sensitivity analysis, OR versus AND
<code>05</code>	Statistical tests, heatmaps, and the qualitative gallery

Alongside every figure panel, each notebook writes a `source_*.csv` containing exactly the numbers drawn in that panel. This is deliberate: any number in any figure can be traced to a file without re-running anything.

5.3 Re-running a notebook safely

Before you run anything, change the output folder in the setup cell to point at your own space:

```
OUTDIR = "/proj/brunk_ecdna_cv_project/YOUR_ONYEN/runs/figures/notebook03"
```

Without this, the notebook rewrites the published figure files in place.

Expect the figures to look slightly different. The plotting library moved from version 3.8 to 3.10 between the original run and now, and default spacing and legend placement changed.

The numbers are unaffected — this has been verified and the evidence is in §8.3. Only appearance changes.

5.4 Which environment should notebooks use?

The `ecdna-bench (canonical)` kernel, always.

There is an older environment called `ecDNA_Hybrid` which was used to draw the original figures. It should not be used for new work:

- It exists only on one person's account and cannot be rebuilt from a file.
- It disappears when that account closes.
- The canonical environment produced every number in the paper, and reproduces the ones `ecDNA_Hybrid` computed — verified numerically in §8.3.

Using one environment for everything means the answer to "how do I reproduce this?" is a single file.

Part 6 — Worked examples

The two questions a reviewer is most likely to ask, and how to answer each with evidence.

6.1 "How do you know it hasn't just memorised these cell lines?"

The full question: *if you train on some cell lines and test on a completely different one, does it still work?*

This is answerable **without changing a single line of code**, because training takes its train/validation assignment from a column in a spreadsheet. Write a new spreadsheet with a different column, point a new settings file at it, and you have a new experiment.

Step 1 — build the modified spreadsheet

```
mkdir -p $MYDIR/runs/loco

cat > $MYDIR/runs/loco/make_split.py <<'PYEOF'
import pandas as pd, os
from pathlib import Path

SRC = Path(os.environ["SOURCE_REPO"]) / "release/manifests" / \
      "dl_master_metadata_stage1_step3_consistency.csv"
OUT = Path(os.environ["MYDIR"]) / "runs/loco"

HOLDOUT = "SNU16"          # the cell line the model must never see

df = pd.read_csv(SRC)
assert HOLDOUT in set(df.cell_line), sorted(set(df.cell_line))

loco = df.copy()
# Every image of the held-out line becomes test, whatever it was before.
# The other lines keep their original assignment.
loco.loc[loco.cell_line == HOLDOUT, "split"] = "test"

print(loco.groupby(["cell_line", "split"]).size().unstack(fill_value=0))
out = OUT / f"consistency_holdout_{HOLDOUT}.csv"
loco.to_csv(out, index=False)
print("\nwritten:", out)
PYEOF

python $MYDIR/runs/loco/make_split.py
```

Read the printed table before going further. The held-out cell line must show **zero** images under `train` and **zero** under `val`. That table is the evidence for the claim, so check it rather than assuming it.

Step 2 — a settings file for this experiment only

```
cd $REPO
sed -e "s|consistency_csv:.*|consistency_csv: $MYDIR/runs/loco/consistency_holdout_SNU16.csv|" \
    -e "s|eccount_out_dir:.*|eccount_out_dir: $MYDIR/runs/loco/eccount_SNU16_holdout|" \
    configs/paths.local.yaml > configs/paths.loco_snu16.yaml

cat configs/paths.loco_snu16.yaml
```

What `sed` does. It reads a file, replaces any line matching a pattern, and writes the result somewhere new. Here it copies your settings file while swapping two lines. The original is untouched.

Step 3 — train

```
cd $REPO
mkdir -p logs
echo $ECDNA_PYTHON           # must not be empty

cp slurm/submit_eccount_train.sh slurm/submit_eccount_train_loco.sh
sed -i "s|configs/default.yaml|configs/paths.loco_snu16.yaml|" \
    slurm/submit_eccount_train_loco.sh

sbatch slurm/submit_eccount_train_loco.sh
queue -u $USER
```

About twelve hours. Watch with:

```
tail -f $MYDIR/runs/loco/eccount_SNU16_holdout/train_history.csv
```

Step 4 — predict on the unseen cell line

Point the settings file at the newly trained model, then run:

```
echo "  eccount_checkpoint: $MYDIR/runs/loco/eccount_SNU16_holdout/best_model.pt" \
    >> configs/paths.loco_snu16.yaml

python -m ecdna_bench.cli.run_eccount \
    --config configs/paths.loco_snu16.yaml --split test
```

Step 5 — score it

```
python -m ecdna_bench.cli.benchmark \
    --config configs/paths.loco_snu16.yaml \
    --models eccount_peaks eccount_threshold \
    --output-dir $MYDIR/runs/loco/benchmark_SNU16_holdout
```

Step 6 — the comparison that answers the question

```
python -c "
import pandas as pd, os
M = os.environ['MYDIR']; S = os.environ['SOURCE_REPO']
pub = pd.read_csv(f'{S}/release/frozen_results/or_matching/summary_by_cell_line.csv')
new = pd.read_csv(f'{M}/runs/loco/benchmark_SNU16_holdout/summary_by_cell_line.csv')
q = lambda d: d[(d.cell_line=='SNU16') & (d.model=='ecount_peaks')].obj_f1.iloc[0]
print(f'SNU16 seen during training : {q(pub):.3f}')
print(f'SNU16 never seen          : {q(new):.3f}')
print(f'difference                 : {q(pub)-q(new):+.3f}')
"
```

The gap between those two numbers is the answer. A small gap means the model learnt what ecDNA looks like in general. A large gap means it learnt what ecDNA looks like *in that cell line*.

Repeat for all four cell lines by changing `HOLDOUT` in Step 1 and re-running. Four independent cluster jobs run at the same time, so the whole experiment is about one day of waiting, not four.

6.2 "Does it work on a cell line you never touched?"

SUM159PT is in the full resource but not in the benchmark, because nobody drew ROI outlines for it. That makes it the strongest generalisation test available: a completely unseen cell line, with real expert annotations to check against.

There is one extra step, because ecCount needs an ROI outline to work inside and SUM159PT has none. A second model predicts those outlines automatically.

Step 1 — predict the outlines. Already done for all 2,986 images; the results are at `$PROJ/Poorya/River/output/all`.

Step 2 — keep only the good ones. The outline model was never trained on SUM159PT and its performance there is *bimodal* — most images are fine, a minority fail badly. So rank the predictions and keep the confident ones:

- by predicted outline area (implausibly small or large outlines are failures)
- by ecDNA retention — what fraction of the expert-annotated dots fall inside the predicted outline

Record how many images you kept and by what rule. That sentence goes in the paper.

Step 3 — run ecCount inside those outlines and score.

Two things that must be stated honestly when this is written up:

1. The outline model was trained on 800 of the 1,145 benchmark images. Any agreement figure quoted on the benchmark is therefore about 70 % training data. Quote the validation-plus-test subset (345 images) for the honest number.
2. When ecCount is evaluated inside a *predicted* outline, the ground truth has been filtered by the same model being evaluated. The result is conditional on the outline rather than end to end, and must be labelled that way. A reviewer will notice if it is not.

Part 7 — When something goes wrong

Errors are normal. Almost every one you will meet is in this list.

How to read an error. Linux error messages put the useful part at the **bottom**, not the top. Scroll to the last few lines first.

command not found

```
bash: pytest: command not found
```

Cause. The environment is not switched on. Every environment has its own set of commands.

Fix.

```
ecdna
```

Confirm your prompt now starts with `(ecdna-bench)`.

ModuleNotFoundError: No module named 'ecdna_bench'

Cause. Either the environment is off, or the code is somewhere it is not expected.

Fix.

```
ecdna
pip install -e $REPO
python -c "import ecdna_bench; print('ok')"
```

No such file or directory: configs/benchmark.yaml

Cause. That file does not exist. Two commands shipped with an out-of-date built-in default.

Fix. Name the config explicitly — this always works:

```
python -m ecdna_bench.cli.benchmark --config configs/default.yaml --output-dir <your folder>
```

Permission denied when writing

Cause. You are trying to write into the published results, which are deliberately locked.

Fix. This is the protection working. Send your output somewhere of your own with `--output-dir`, or change `results_root` in `configs/paths.local.yaml`.

Jupyter will not start, or the kernel dies immediately

Cause. Nine times out of ten, a full home folder. Jupyter cannot write its session files and gives up, usually with an error mentioning nothing about disk space.

Fix.

```
quota -s
du -sh ~/.cache ~/.local ~/.conda 2>/dev/null
```

If home is near its limit, confirm the cache lines from Step 4 are in your `.bashrc`, then clear what has already accumulated:

```
rm -rf ~/.cache/pip
conda clean --all --yes
```

The kernel appears in the menu but never connects

Cause. The notebook engine is a newer version than the Jupyter server expects.

Fix.

```
ecdna
pip install "ipykernel<7"
```

Then restart the Jupyter session in OnDemand.

A job fails instantly with `ModuleNotFoundError: No module named 'ecdna_bench'`

Cause. `ECDNA_PYTHON` is not set, so the job used the system Python instead of the project environment. This is the single most common cluster failure here.

Fix.

```
echo $ECDNA_PYTHON
```

If empty, confirm the line is in your `.bashrc` (Step 4 of Part 3), then `source ~/.bashrc` and resubmit.

A job fails instantly with nothing in the log at all

Cause. The `logs/` folder does not exist. SLURM cannot create the log file, so it gives up before running anything.

Fix.

```
mkdir -p $REPO/logs
```

```
sbatch: error: invalid partition specified
```

Cause. Partition names change when a cluster is upgraded.

Fix.

```
sinfo -s
```

Copy the job script and update its `#SBATCH -p` line to a partition that exists.

A job ran, but wrote into someone else's folder

Cause. You submitted from the original repository instead of your own. The scripts take the project root from the folder you submitted from.

Fix. Check with `pwd` before every `sbatch` — the path must contain your own ONYEN. Or state it explicitly:

```
ECDNA_PROJECT_ROOT=$REPO sbatch slurm/submit_benchmark.sh
```

A cluster job disappeared with no output

Fix. Read the log files. The real error is almost always in `.err`, not `.out`:

```
squeue -u $USER
sacct -j JOBNUMBER --format=JobID,JobName,State,Elapsed,ExitCode
seff JOBNUMBER
ls -lt $MYDIR/runs/logs | head
tail -50 $MYDIR/runs/logs/<the newest .err file>
```

Reading the outcome:

State	Meaning	Fix
COMPLETED, ExitCode 0:0	Success	—
OUT_OF_MEMORY	Ran out of memory	Increase <code>--mem</code> in the job file
TIMEOUT	Hit the time limit	Increase <code>-t</code>
FAILED	The program itself errored	Read the <code>.err</code> file

`torch.cuda.is_available()` returns `False`

On a login node this is correct — login nodes have no graphics card.

On a compute node, check the job actually asked for one: `--partition=a100-gpu --qos=gpu_access --gres=gpu:1`

The numbers do not match the paper

In order of likelihood:

1. **Wrong matching folder.** `or_matching/` and `and_matching/` contain files with identical names. **OR is the published one.**
2. **Wrong subset.** The paper quotes both the pooled benchmark (1,145 images) and the test split alone (175). Check which table you are reading.
3. **Wrong averaging.** Pooled F1 sums all hits and misses first, then computes the score once. Per-image-mean F1 scores each image separately and averages. These differ — 0.942 against 0.931. **Pooled is the published one.**

A job crashed the machine while processing many images

Cause. Code that asks how many processors the machine has and starts that many workers. SLURM gave you eight; the machine has seventy-two. The code starts seventy-two and runs out of memory.

Fix. Use a simple loop that saves progress to a file periodically. Slower per image, and it survives.

I have broken my `.bashrc` and nothing works

Fix. Restore the backup from Step 4:

```
cp ~/.bashrc.backup-YYYYMMDD ~/.bashrc
source ~/.bashrc
```

If you have no backup, log in and run `bash --norc` to get a working shell with no configuration, then edit the file with `nano ~/.bashrc`.

I have overwritten a results file

Fix. Git has it.

```
cd $REPO
git status
git restore path/to/the/file
```

`git status` lists what changed; `git restore` puts it back exactly.

Part 8 — Reference

8.1 Glossary

Term	Meaning
ONYEN	Your UNC username
Terminal	The text window you type commands into
Bash	The program that reads and runs what you type
.bashrc	A file bash reads automatically at startup
Environment variable	A named shortcut, e.g. <code>\$PROJ</code>
Path	The address of a file or folder
Login node	Where you land. Editing and submitting only
Compute node	Where real work runs, allocated by SLURM
SLURM	The scheduler that hands out compute nodes
Partition	A group of compute nodes (<code>general</code> , <code>a100-gpu</code> , ...)
Conda environment	A self-contained set of Python libraries at fixed versions
Kernel	The engine a notebook uses to run code
Git commit	A permanent snapshot of the project
YAML	A human-readable settings file format
ecDNA	Circles of DNA outside the chromosomes, carrying oncogenes
FISH	Fluorescence In Situ Hybridisation — makes chosen DNA glow
Metaphase spread	One cell's DNA, arrested mid-division and spread flat
ROI mask	An outline marking which spread to analyse
Ground truth	Where a human said each ecDNA is
Object F1	Detection accuracy after matching predictions to truth
Precision	Of what you reported, how much was real
Recall	Of what was real, how much you found
Count MAE	Average size of the counting error
Signed bias	Average error keeping its sign — does it lean one way
Pooled / micro-averaged	Sum everything first, then score once. Published
Per-image-mean	Score each image, then average. Not published
OR / AND matching	Whether a match needs distance <i>or</i> overlap, or both. OR is published
Anchor image	<code>ncih2170_facsc_fish_0723_low_her2_52</code> , 116 ecDNA — the running example

8.2 The published numbers

Pooled, OR matching, all 1,145 benchmark images:

Method	Object F1	Count MAE	Signed bias
ecCount (peaks)	0.942	13.1	+0.4
ecCount (threshold mask)	0.917	18.1	−11.4
Label Engine	0.825	36.0	−28.8
MIA	0.800	47.0	−40.6
Classic (after opt)	0.777	44.9	−24.3
ecSeg	0.464	121.5	−118.5

Test split only (175 images):

Method	Object F1	Count MAE
ecCount (peaks)	0.939	13.3
ecCount (threshold mask)	0.916	17.6
Label Engine	0.825	34.4
MIA	0.813	40.4
Classic (after opt)	0.775	40.9
ecSeg	0.508	110.3

Resource: 2,986 images, 5 cell lines. **Benchmark:** 1,145 images, 4 cell lines, split 800 / 170 / 175. **Model:** 7,849,601 parameters, best at epoch 49, validation loss 0.6421. **Matching settings:** `d_max = 20 px`, `IoU_min = 0.1`, `alpha = 0.5`, OR policy, 8-connectivity, minimum object area 3 px.

Method names, spelled exactly this way everywhere: `ecCount (peaks)` · `ecCount (threshold mask)` · `Label Engine` · `MIA` · `Classic (after opt)` · `ecSeg`

8.3 Environment provenance

Which environment produced which artefact:

Artefact	Produced by	Environment
Every published metric	Cluster jobs	<code>ecdna-bench</code>
Trained model, training history	Cluster job	<code>ecdna-bench</code>
Figure files, notebooks 01–05	Jupyter	<code>ecDNA_Hybrid</code>

Measured difference between the two, 30 August 2026:

Package	ecdna-bench	ecDNA_Hybrid	Consequence
python, numpy, pandas, scikit-learn, Pillow, torch, seaborn	identical	identical	none
scikit-image, imageio, tifffile	0.25.2 / 2.37.x / 2025.5.10	effectively the same	none
scipy	1.15.3	1.13.1	the only gap that could move a value
statsmodels	0.14.6	0.14.4	patch level only
OpenCV	4.13.0	4.10.0	classical pipeline only, which ran in ecdna-bench
matplotlib	3.10.0	3.8.4	figure appearance only

Verified. The paired statistical tests reported in the paper were recomputed from the released per-image data under both SciPy versions. The two runs were identical to each other and to the committed values:

Subset	n	W (two-sided)	p (two-sided)	W (greater)	p (greater)
All images	1,145	118.0	1.50×10^{-188}	654,822.0	7.49×10^{-189}
Test only	175	3.0	1.90×10^{-30}	15,397.0	9.52×10^{-31}

The canonical environment was also confirmed self-contained: every library resolves inside the environment folder, with versions matching [env/environment.yml](#) exactly. Nothing depends on packages in a personal home directory.

Conclusion. [env/environment.yml](#) reproduces the published results. [ecDNA_Hybrid](#) is not needed and should not be used for new work.

8.4 Where things are in the repository

```

ecdna-bench/
├── configs/
│   ├── default.yaml          every published setting ☐ never edit
│   ├── paths.example.yaml    template for local paths
│   └── paths.local.yaml       yours; not shared
├── env/environment.yml        the environment recipe
├── src/ecdna_bench/
│   ├── cli/                  every command you can run
│   ├── data/                 catalogue, file handling
│   ├── classical/            traditional pipeline and its tuning
│   ├── eccount/              the neural network
│   ├── converters/           converters for the external tools
│   ├── evaluation/           matching, metrics, statistics
│   └── roi/                   the outline-prediction model
├── slurm/                     cluster job files for every stage
├── scripts/                   helper tools
├── notebooks/                 01☐05, produce every figure
├── release/
│   ├── frozen_results/       or_matching/ ☐ published ☐ and_matching/
│   ├── figures/notebookNN/    figure files plus their source numbers
│   ├── split_files/           which image is in which split
│   ├── manifests/             catalogues and file checksums
│   └── model_checkpoints/      the published trained model
├── docs/                      further documentation
└── tests/                     automated checks ☐ 124 passing

```

8.5 Command card

```

# Start any session
ecdna

# Where am I, is anything broken?
pwd
git status
queue -u $USER

# Cluster jobs – always from the repository root, with logs/ existing
cd $REPO && mkdir -p logs
sbatch slurm/submit_benchmark.sh          # score everything
sbatch slurm/submit_eccount_infer.sh       # run the model on images
sbatch slurm/submit_eccount_train.sh       # train from scratch, ~12 h

# Change behaviour without editing a file
MODEL=eccount_peaks SPLIT=test sbatch slurm/submit_sensitivity.sh

# Score by hand, into a folder of your own
python -m ecdna_bench.cli.benchmark \
  --config configs/default.yaml \
  --output-dir $MYDIR/runs/benchmark_$(date +%Y%m%d)

# Check on a job
queue -u $USER
sacct -j JOBID --format=JobID,State,Elapsed,ExitCode
seff JOBID

# Undo a mistake
git restore <file>

```

The three rules: Read from shared folders, write only to your own · `or_matching/` is the published one · Pooled F1, not per-image-mean.

8.6 Getting help

Cluster problems — accounts, quotas, jobs, storage: <https://help.rc.unc.edu>

Project questions — the science, the numbers, the code: the Brunk Lab.

When asking about an error, include: the exact command you ran, the last twenty lines of the error, and the output of `squeue -u $USER` if it was a cluster job. That is almost always enough to diagnose it immediately.